

place_pins

NAME

place_pins
Places pins on blocks within the current design or on the current block.

SYNTAX

status place_pins
[-cells cells]
[-nets nets]
[-pins pins]
[-ports ports]
[-exclude_nets nets]
[-nets_to_exclude_from_routing nets]
[-self]
[-legalize]
[-use_existing_routing]

Data Types

cells	collection
nets	collection
pins	collection
ports	collection

ARGUMENTS

- cells cells
Specifies the blocks for pin assignment. Only the nets connected to the specified cells are routed during global routing and only pins on selected blocks are placed. By default, the command performs pin assignment on all blocks.
- nets nets
Specifies the nets for pin assignment. Only the specified nets are routed during global routing stage and only block pins connected to the selected nets are placed. By default, the command performs pin assignment on all nets.
- pins pins
Specifies the pins for pin assignment. Only the specified pins are placed when this option is used. The tool routes the nets that are connected to the specified pins but the tool skips feedthrough creation on those nets. So no feedthrough nets or pins are created or placed. By default, the command performs pin assignment on all block pins.
- ports ports
Specifies the top-level ports for pin assignment. The command places only the terminals associated with the specified ports. By default, the tool performs pin assignment on all block pins.
- exclude_nets nets
Specifies the nets that are excluded for pin assignment and feedthrough creation. All nets, except the excluded nets, are routed and pins are placed based on the routed nets. All the pins including feedthrough pins are placed except the pins that connected to those specified nets.
- nets_to_exclude_from_routing nets
Specifies the nets that are excluded from routing during pin placement. By default, the tool tries to route all the nets that are used for pin placement. This option can be useful if some nets are preferred to skip routing to get quick pin placement results on them. When this option is specified, the place_pins command performs pin placement based on net connectivity to minimize wire length. This option cannot be used with the option -use_existing_routing , since the routing is skipped when using that particular option.
- self Performs pin placement on the top-level design. By default, pin placement is performed on blocks within the current top-level design.
- legalize
Performs only pin legalization. The command runs in incremental mode(the value of app option plan.pins.incremental is ignored) and legalizes pin overlaps, pin shorts with pre-routes and off-

track pins. Routing of nets is skipped, this can be used to quickly legalize existing pin placement. With this option pins that are already legal are not moved and pin shapes are not created for this pins/ports that do not already have shapes associated. That is only existing pins are legalized if needed. Existing pin shapes are considered legal if they are on edge, on track, do not have any overlaps and do not violate any technology related spacing constraints (like layer minimum spacing). Note that the app option to synthesize abutted pins is ignored during the legalize mode of the command.

This option cannot be used with the `-use_existing_routing` or the `-nets_to_exclude_from_routing` options.

`-use_existing_routing`

Uses existing global routing when placing pins. When this option is specified, the command `place_pins` expects that the nets of the relevant pins are routed by proper global routing commands already, for example, `route_global -virtual_flat` or `route_group -global_planning true`. These routing results will be removed after `place_pins` completes.

When this option is specified and no routing exists for a net, the locations of the relevant pins might not be optimal. The command `place_pins` will not try to route those nets. It will not calculate the flyline based location either.

Note that when running `global route` before `place_pins -use_existing_routing`, if the block pins already exist in the design, then the global router will route to those pre-existing pins. As a result, if pins are then placed by using these existing routing, their locations might have little or no changes.

Note that this option cannot be used with the `-pins` option.

DESCRIPTION

This command places pins on blocks within the current design, or the top ports of the current design, based on the command line options specified by user. If user does not specify the types of pins to be placed, it places pins for all the editable blocks without fixed physical pins within the current design.

The command first performs global routing, then places pins in legal locations on wire tracks based on the global routing results. The `place_pins` command does not use reserved tracks for pin placement. Depending on the type of pins to be placed, the wire tracks used are:

- o For the top ports, `place_pins` uses wire tracks defined at the top.
- o For the block pins, `place_pins` uses wire tracks defined inside the corresponding blocks. If the corresponding block does not have any wire tracks defined, then `place_pins` will use the wire tracks defined at the top.

Pin placement also honors pin constraints set by the `create_topological_constraints`, `set_individual_pin_constraints`, `set_bundle_pin_constraints`, and `create_block_pin_constraint` commands. The constraints set by those commands are therefore referred to as topological pin constraints, individual pin constraints, bundle pin constraints, and block pin constraints, respectively. These four types of pin constraints can also be defined in a pin constraints file, which can be read in by command `read_pin_constraints`.

If multiple pin constraints are applied to the same pins, normally the constraints are applied with the following priority:

- o topological pin constraints
- o individual pin constraints
- o bundle pin constraints
- o block pin constraints

However, for bundle pin constraints, if the option `-keep_pins_together` is set true, then `place_pins` will try to honor it whenever possible. In some cases, it might have to relax the individual pin constraints requested layer or spacing.

Certain types of pin constraints, e.g. spacing, location, layer, or pin length, can be specified as hard constraints by command `create_block_pin_constraint`. When these constraints are hard, `place_pins`

honors the applicable value and skip all DRC rule checks, even if it means possible DRC violations. For example, if location is hard and one pin is constrained at a location inside pin blockage, or will be shorted to another pin, this pin will be placed at the location regardless. Or if length is hard and one pin is constrained by a pin length that is shorter than minimum length, or this value violates min area rule, the pin will still be of the specified length regardless these violations.

A pin guide or a routing corridor can be created to refrain certain pins within the defined region. Alternatively, a pin blockage can be created to forbid certain pins within the defined region.

If pin constraints are so restrictive that place_pins cannot honor all of them, one or more constraints might be relaxed to place the constrained pins, provided they are not hard constraints. Depending on the type of constraint a pin has, the sequence of constraints relaxation can be slightly different.

If the pin has the side and offset constraints, the constraints relaxation sequence is the following:

- o If the pin has a non-zero range constraint such as offset range, pin guide, routing corridor, etc, then relax the spacing. Otherwise, relax the offset.
- o Relax both offset and spacing.
- o Ignore the edge abutment.

If the pin does not have the side and offset constraints, the constraint relaxation sequence is the following:

- o Relax spacing.
- o Ignore edge abutment but honor spacing.

If the preceding constraints relax sequence cannot find a solution to place the pin, the command tries additional relaxation sequence as follows:

- o Relax offset but honor others.
- o Relax layer but honor others.
- o Relax spacing but honor others.
- o Relax offset, layer and spacing.

During constraint relaxation, if any of the constraints are hard constraints, the corresponding relaxation is skipped.

The pin width, length and spacing can be affected by other factors in addition to the pin, bundle or block pin constraints, if they are not specified as hard constraints.

In particular, the pin width is decided by the applicable pin constraints applied to them according to the priority sequence above, with the following exceptions:

- o if the pin is placed on a wire track with non-zero track width, then the pin width is the same as the track width. If the track width is less than the default width of the layer, the width is then promoted to the default width. All other width related pin constraints are ignored.
- o if the pin has no width-related pin constraints, but its net has a non-default routing rule requiring a width on the layer, then the pin width is same as the non-default rule specified width
- o if the pin has no width-related pin constraints and no non-default routing rule, then the pin width is the same as the maximum of the default width and min-width rule defined by the technology file

The pin length is decided as the maximum of the pin length constraints (if applicable/present) and the following:

- o the pin width
- o the min-length rule defined in the technology file on the layer if it is applicable

- o the pin-length required to satisfy the min-area rule if there is a min-area rule defined in the technology file on the layer. Note that width based min-area rules are also supported, the maximum of the min-area rule and width-based min area rule will be applied if both are present.

- o lastly, if the layer has the mask constraints defined, the pin length is promoted to 10X routing pitches

Note that the length of the pins after being determined as the maximum of the above mentioned constraints/values, is rounded up to the nearest multiple of the grid size (if grid is present). This is to ensure that pins are created on the manufacturing grid. The depth is rounded to nearest multiple of grid size that is greater than constrained depth.

If there is no individual or bundle pin spacing constraints, then the pin spacing is decided by the following:

- o if its net has a non-default routing rule requiring a spacing distance on the layer, then the pin spacing to the nearest pin honors the specified distance

- o if its net does not have a non-default routing rule or the non-default routing rule does not have a spacing distance requirement on the layer, then the pin spacing is decided by the command `create_block_pin_constraint`

The `set_editability` command can enable or disable this command for blocks or levels of physical hierarchy. The `place_pins` command has no effect on the blocks that are disabled by `set_editability`.

If feedthrough creation is allowed, this command might create feedthrough nets and ports, depending on the global routing results. If feedthroughs are created, the design netlist is modified. The newly-created feedthroughs are automatically updated with multivoltage (MV) constraints if the design uses UPF.

If feedthroughs pins/nets are created, by default, newly created feedthrough ports and nets are named `"net_name_FEEDTHRU #"` in `place_pins`, where `net_name` is the name of the original net that connected this feedthrough port or feedthrough net, and `"#"` is an index number. The names of newly-created feedthroughs pins/nets can be controlled by following application options:

```
plan.pins.new_port_name_tag
plan.pins.new_port_name_tag_style
```

Use the following application option to control the split-net naming convention:

```
set_app_options -name plan.pins.new_split_nets_use_block_names -value true
```

When this application option is true, the tool inserts the feedthrough master block names into the feedthrough net name. The inserted string starts with master name of the driving block, then a sequence of all the load blocks master names separated by double underscore characters. By default, this application option is false by default and block names are not inserted into the feedthrough net name.

For example, if you are using suffix name tag style (default), the feedthrough net name is:

```
BaseNetName + "__" + BlockBasedSubString + "__" + NameTag + __
IndexNumber
```

For prefix name tag style:

```
NameTag + "_" + IndexNumber + "__" + BaseNetName + "_" + BlockBased-
SubString
```

Note that this only applies to split nets that connect one block to one or more other blocks. It DOES NOT apply to any split net that has a top port connection in the block in which the split net belongs.

You can disable pin placement on an a previously-placed pin by setting the `physical_status` to "fixed" for the terminal of this pin. The following command sets this "fixed" status on terminal of pin block/A and pin placement skips placing this pin.

```
prompt> set_attribute [get_terminals -of_objects [get_pins block/A]] physical_status fixed
```

By default, pin placement tries to generate the global route and place

pins so that block pins can be aligned for nets connecting two pins. To direct the tool to apply more effort to achieve better pin placement, use the `plan.pins.reserved_channel_threshold` application option.

By default, clock nets are excluded from feedthrough creation. To enable feedthrough creation on clock nets, set the `plan.pins.exclude_clocks_from_feedthroughs` application option to false.

When `-self` option is specified, the tool still places pins for the ports that are not marked as power/ground type but connected to power/ground net. When `-self` option is not specified and block pins are connected to top level power/ground net but the block port type are not power/ground, the tool still places pins for those block ports.

Users can use app options to control pin placement behavior. Specifically, pin placement related app options start with `plan.pins`. The following are current app options related to `place_pins`. For more details, use `report_app_options plan.pins*` to list those app options and their current settings.

```
plan.pins.allow_pins_in_narrow_macro_channels
plan.pins.exclude_clocks_from_feedthroughs
plan.pins.exclude_inout_nets_from_feedthroughs
plan.pins.incremental
plan.pins.layer_range
plan.pins.new_port_name_tag
plan.pins.new_port_name_tag_style
plan.pins.new_split_nets_use_block_names
plan.pins.pin_range
plan.pins.reserved_channel_threshold
plan.pins.retain_bus_name
plan.pins.synthesize_abutted_pins
plan.pins.ignore_pin_spacing_constraint_to_pg
plan.pins.treat_pg_as_shield
plan.pins.enable_span_rules
```

If there are mask constraints defined on the tracks and if `place_pins` creates any pins on the mask constraints defined tracks, the mask constraints will also be set on the pins as the following:

- o The mask constraint set on the pin is from one of the values set on the tracks. `place_pins` will not use any other values not seen on the current tracks.
- o `place_pins` first checks the pins nearest usable tracks. A usable track is the track that is not occupied by this pin, and if there is a layer shape on this track with minimum width, it does not trigger the minimum spacing or any mask constraint spacing violation. The nearest usable tracks are those usable tracks on this pins left and right (or top and bottom) that are nearest.
- o Then `place_pins` negates the mask constraints values on those nearest usable tracks. If there are common values between the negated mask constraints values on the left (or bottom) nearest usable track and the right (or top) nearest usable track, `place_pins` picks one value and applied it to this pin.
- o If there are no common values between the negated mask constraints on the left (or bottom) nearest usable track and the right (or top) nearest usable track, `place_pins` chooses the mask constraint on the center track that this pin occupies and apply it to this pin. And a warning message shall be issued to the user that no applicable mask constraints values could be found.

This command returns 1 on success, 0 otherwise.

EXAMPLES

The following command places pins on all blocks within the current design.

```
prompt> place_pins
```

The following commands enable four cores for pin placement and perform pin placement on all blocks within the current design.

```
prompt> set_host_options -max_cores 4
prompt> place_pins
```

SEE ALSO

[route_global\(2\)](#)
[create_block_pin_constraint\(2\)](#)
[set_bundle_pin_constraints\(2\)](#)

[set_editability\(2\)](#)
[set_host_options\(2\)](#)
[set_individual_pin_constraints\(2\)](#)
[write_shadow_eco\(2\)](#)

Version V-2023.12-SP5-6
Copyright (c) 2025 Synopsys, Inc. All rights reserved.

Man(1) output converted with [man2html](#)